॥ श्रीगणेशाय नमः ॥

# Deep multi-layer perceptrons

## A brief history

Around 1980s to around 2010's, people were trying to build 2-3 layered networks.

Main problems:
- vanishing gradient
- too little data (easy overfitting)
- too little compute (mini-batches are used, so time would be too much).

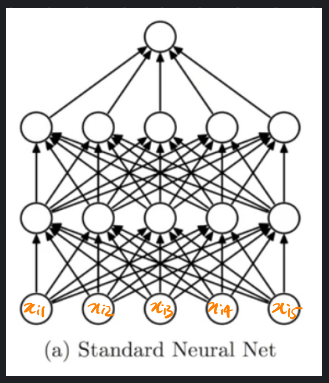
Upto 2010:
- there was a lot of data (due to the internet)
  (labelled) competitions like ImageNet took place.
- GPUs
- new algos

## Dropout and regularization

Consider random forests. We sample a subset of columns. We build many trees, each only looking at a subset of features.
Then we take a majority vote.
So we use randomization to achieve regularization (we reduce the overfitting).

(fully grown DTs) → overfit model (low bias, high variance)

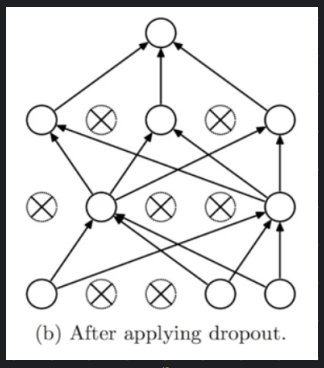
A similar method is applied to NN (dropouts).



(a) Standard Neural Net

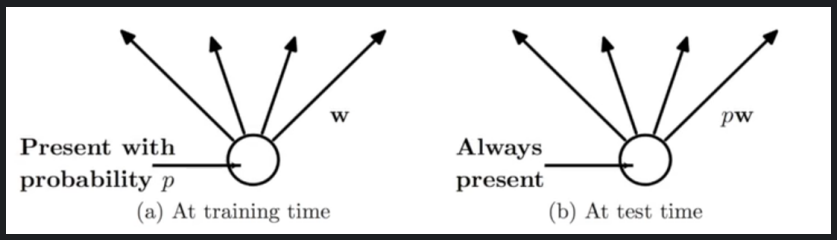
At any given iteration in any layer, we randomly take a bunch of neurons, and remove all the connections to and from them (for that iteration only).

Dropout rate
$0 \leq p \leq 1$
↳ prob. of a neuron being inactive (dropped out)


(b) After applying dropout.

$p \approx 0.5$

Drop-out, in idea, is similar to randomly subsetting some features.



(a) At training time (b) At test time

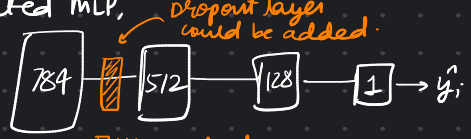
A unit is only dropped at training.

At test, the weights are reduced to $pw$ as the neuron was present only with a probability $p$. ← Here $(1-p)$ is dropout.

If there are more weights than data points, overfitting ↑↑, then keep $p$ (dropout rate) smaller.
(really deep NN) (say 0.1/0.2)
(or you could even use CV)

Dropouts are often thought of as extra layers in a NN.

For a fully connected MLP,




It'll randomly filter out values from the previous layer.

# Rectifier (ReLU)

$$(w_{ij}^k)_{new} = (w_{ij}^k)_{old} - \eta \frac{\partial L}{\partial w_{ij}^k}$$

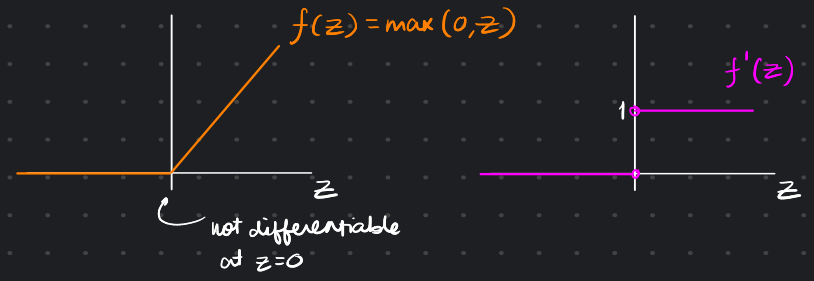
this gets really small ⇒ vanishing gradient
convergence is harder.

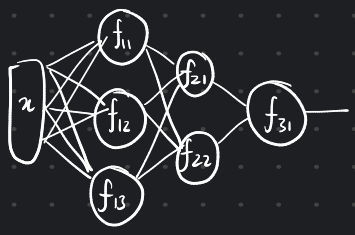
we cannot train really deep neural networks with sigmoid/tanh activation functions.

ReLUs are instead used in MLPs.

$$f(z) = z^+ = max(0, z) = \begin{cases} 0 & \text{if } z \le 0 \\ z & \text{otherwise} \end{cases}$$

$$= \frac{z + |z|}{2}$$




Problem was the multiplicar of derivatives less than 1.
Now derivatives are either 0 or 1 (neither less, nor more).

$$\frac{df_{ReLU}}{dz} \in \{0, 1\}$$

so there's no vanishing/exploding gradient.

0 causes dead activations though.

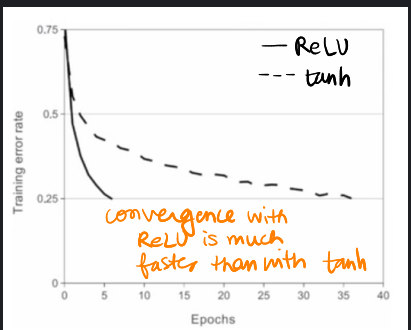
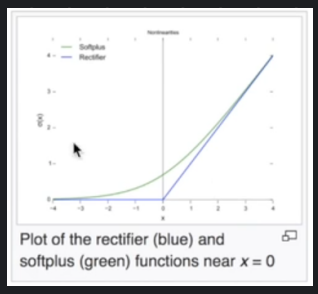



Figure 1: A four-layer convolutional neural network with ReLUs **(solid line)** reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with tanh neurons **(dashed line)**. The learning rates for each network were chosen independently to make training as fast as possible. No regularization of any kind was employed. The magnitude of the effect demonstrated here varies with network architecture, but networks with ReLUs consistently learn several times faster than equivalents with saturating neurons.

A smoother approximation to the rectifier is the softplus function.

$$f(x) = \log(1 + \exp x)$$

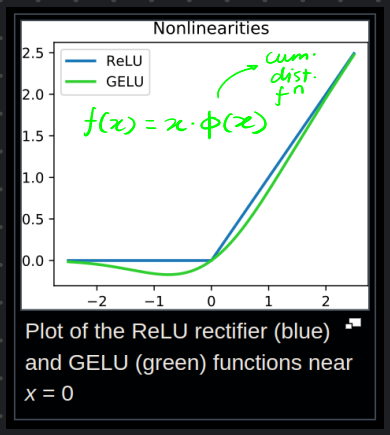

Plot of the rectifier (blue) and softplus (green) functions near $x = 0$

These smoother approximations have good derivatives.

For softplus,

$$f'(x) = \frac{1}{1 + e^{-x}}$$ (logistic sigmoid)

Another is the Gaussian-error linear unit.



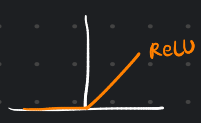
Plot of the ReLU rectifier (blue) and GELU (green) functions near $x = 0$

Noisy ReLU: Gaussian noise is included.

$$f(x) = max(0, x + Y)$$

$Y \sim N(0, \sigma(x))$ Gaussian noise

Problem with ReLU:




If $z < 0$, $f(z) = 0$
$f'(z) = 0$

In chain rule, we multiply a lot of derivatives. Even if one of them is zero, the whole thing $\frac{\partial L}{\partial w_{ij}^k}$ becomes zero.

The update func$^n$ $w_{ij}^k{}_{new} = w_{ij}^k - \eta \underbrace{\frac{\partial L}{\partial w_{ij}^k}}_{0}$ doesn't update the weights.

For any neuron, if all inputs are normalized, and weights are -ve, then $z = w^T x$ becomes -ve if too many neurons die.

Say we have like 20 activ$^n$ units and 15 are dead. The model doesn't learn anything.

A simple fix is leaky relu.

## Leaky ReLU




$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{otherwise} \\ \hookrightarrow ax & \end{cases}$$

↳ can be treated as hyperparameter

Again, there's the added chance of vanishing gradients as $a$ is really small.

$$\frac{df_{LR}}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ a & \text{otherwise} \end{cases}$$

Problems with ReLU:

- Non-differentiable at zero.
- non-zero centred
- unbounded: as $z \uparrow$ $f(z) \uparrow$ (there's no bound)
- dying ReLU: fix: Leaky ReLU

Advantages:

- speeds up convergence
- easy to compute $f(z)$ and $\frac{\partial f}{\partial z}$ ← a simple if-else conditions.

If wts are properly initialized, dying ReLU problem is reduced a lot.

## Weight initialization

In logistic regression, weights in SGD are initialized randomly, or $W_{ij}^{k} \sim N(0, \sigma)$.

(Bad) Idea 1: init $W_{ij}^{k} = 0 \quad \forall i, j, k$ Really bad idea.

$f_{ij}$'s will all be the same.

Every neuron will get input $W^T x = 0$ throughout the network.

No "learning happens".

Problem of symmetry { - all neurons compute the same thing. - same gradient updates happen. } also if all wts are initialized to the same value.

So we want asymmetry.

In ensemble learning as well, more diff base models give better output for ensembling.

(Bad) Idea 2: Initialize $W_{ij}^{k}$s as large -ve #s.

$W^T x = z$ become large -ve values. we face the dying ReLU problem. $f(z) = 0$.

↑ normalized (mean centering, variance scaling)

So $W_{ij}^{k}$s should NOT be large -ve #s.

Idea 1: weights should be small (but not too much)

not all zeros (or all same)

good variance (lot of variability means each model will learn differently)

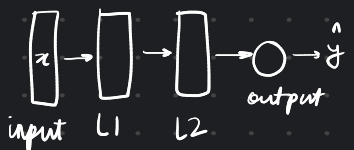
$W_{ij}^{k} \sim N(0, \sigma)$ for a small $\sigma$.

Gaussian initializ$^n$.

# Batch normalization

Problem:

consider a fully-connected MLP.




As a pre-processing step, we do data normalization on the $x_i$'s.

↳ mean & variance
0 1

$$D = \{x_i, y_i\}$$

$$\tilde{x}_i = \frac{x_i - \mu}{\sigma}$$

For a really deep network,

$x - L_1 - L_2 - L_3 - L_4 - L_5 - \text{output} - \hat{y}$

say the input is in mini-batches.

small change here

large change here

A small change in the input can lead to a large change in the later layers.

Consider a small square operation.

1 → 1
1 → 1.01
1 → 1.0201
1 → 1.0406

after 5 times

1 → 1.375

↳ ≈ 37% more than the initial value.

So for operations like in ReLU, small changes anywhere would lead to huge changes in the deep layers.

$x - L_1 - L_2 - L_3 - L_4 - L_5 - \text{output} - \hat{y}$

$\{\} \rightarrow$ mini batch $\rightarrow$

consider a neuron here

It gets a batch input.
The neuron won't see much diff in 2 batches as the data is normalized.

consider a neuron here

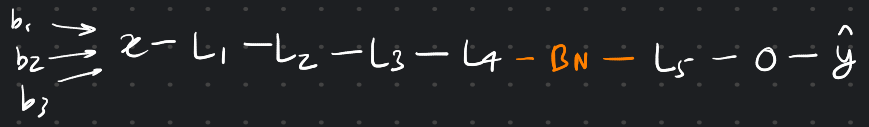
After tons of operations, there'll be much diff betn 2 batches.
Very dissimilar batches.

If you think of a layer as a model, ideally a model should get normalized data.
But the distributions are very diff. for diff batches.

Internal covariance shift.

Solution:

Add a new batch normalization layer.




We explicitly normalize each layer. so the distribution of batches is similar.

**Input:** Values of $x$ over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: $\gamma, \beta$
**Output:** $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

**Algorithm 1:** Batch Normalizing Transform, applied to activation $x$ over a mini-batch.

$\mathcal{B} \rightarrow$ [BN, $\gamma,\beta$] $\rightarrow \tilde{\mathcal{B}}$

$\{x_i\}_1^m$ $\qquad$ $\{\tilde{x}_i\}_1^m$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

$\epsilon$: small val to prevent div. by zero error

$\gamma, \beta$ will be learned through backpropogn.

$$\tilde{x}_i = \gamma \hat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i)$$

Advantages:

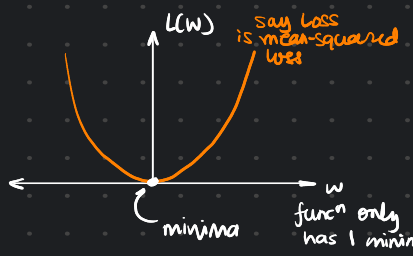
- Faster convergence: Inputs from same distribn, so we can afford to have a larger learning rate.
- Also works as a weak regularizer. (But still use dropouts)
- Avoids internal covariance shift: so can train deeper neural networks.

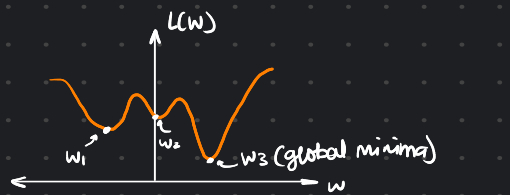
# Optimizers: hill descent

## Terminology:

Optimization problem: $\min_w L(w)$

case a: w is a scalar.




func^n has 3 local minimas of which 1 is global.

Saddle point:

At saddle point as well, $\frac{dL}{dw} = 0$

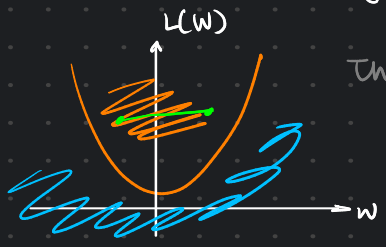
consider diff loss func^n's.

At any minima/maxima, $\frac{dL}{dw} = 0$

So when we get a zero slope at any point, the point could be → maxima / minima / saddle point

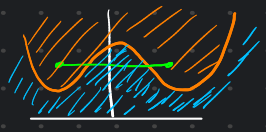
So SGD could get stuck (converge) at saddle points.

## convex functions:




The func^n divides the plane into 2 points.

If for any 2 points in the same region, if line connecting them lies entirely in the same region ⇒ convex function.



⇒ non-convex

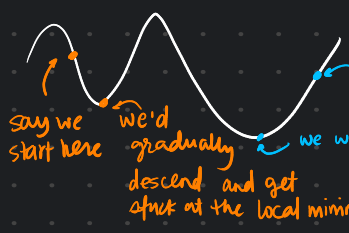
convex functions only have 1 maxima/minima. So local minima = global minima.

non-convex func^n's have > 1 minima/maximas.

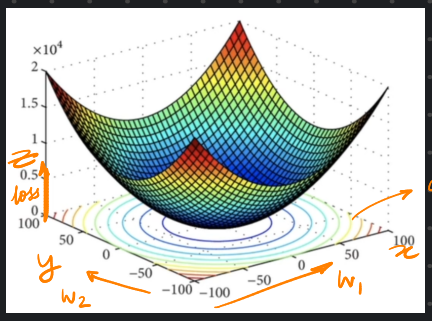
func^n's like linear regression, SVM, logistic regressions, loss func^n's are convex.
For deep learning, loss func^n's are non-convex.

---

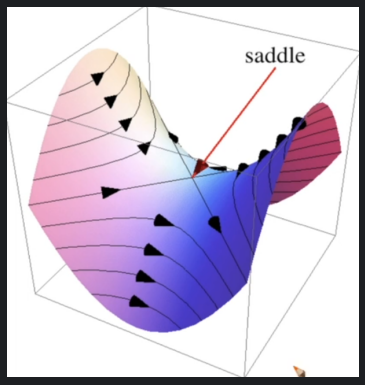
optimiza^n is like descending from a hill (the loss func^n)



Say w is 2D:




contour lines
(all points at same height are plotted and projected on 2D plane)



# Stochastic gradient descent

$$(w_{ij}^k)_{new} = (w_{ij}^k)_{old} - \eta \left[\frac{\partial L}{\partial w_{ij}^k}\right]_{(w_{ij}^k)_{old}}$$

let's make notn simpler:

$$w_t = w_{t-1} - \eta \left[\frac{\partial L}{\partial w}\right]_{w_{t-1}}$$

If $\frac{\partial L}{\partial w}$ is computed using all the points in the dataset → GD

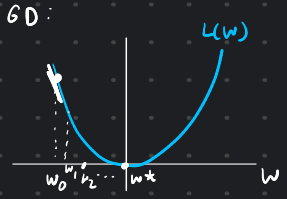
→ only 1 pt. $x_i$ randomly chosen from D → SGD

→ a subset of k points in D → mini-batch SGD. } used mostly.

The derivatives when mini-batch / SGD is used is a little bit noisy. A reasonable batch size gives a good approxmn though.

---

GD:




convergence with GD

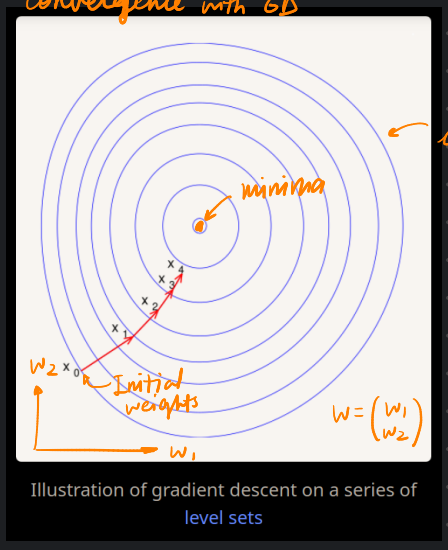
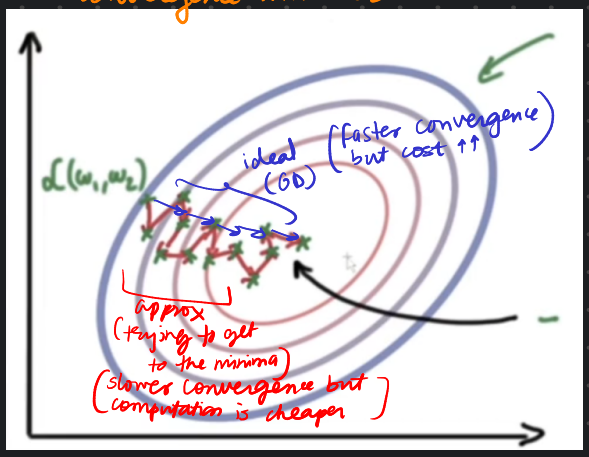



Illustration of gradient descent on a series of level sets

convergence with SGD:




SGD is an approximation to GD.

We won't be able to exactly follow the best path to the minima like in the above diagram.

But we'll eventually get there.

We try to estimate the gradient value in SGD as opposed to GD where we calculate the actual value.

Each of the updates are more noisy in mini-batch SGD than in GD.




We can try and de-noise the values.

# SGD with momentum

A simple de-noising strategy:

Consider a list of #s that we get at diff values of t:

| t=1 | t=2 | t=3 |
|---|---|---|
| $a_1$ | $a_2$ | $a_3$ |

A simple de-noising trick is to take averages.

$t=1$ $\quad v_1 = a_1$

$t=2$ $\quad v_2 = \gamma v_1 + a_2 \qquad \gamma \in [0, 1]$

If $\gamma = 1$, $v_2 = v_1 + a_2 = a_1 + a_2$ — It's like a weighted sum.

If $\gamma = 0$, $v_2 = a_2$

$$v_3 = \gamma v_2 + a_3$$
$$= \gamma(\gamma v_1 + a_2) + a_3$$
$$= \gamma^2 v_1 + \gamma a_2 + a_3$$
$$= \gamma^2 a_1 + \gamma a_2 + 1 \cdot a_3$$

something lesser (as $\gamma \in [0,1]$) — something even smaller — 1

so as we go further, the previous values are given a lesser weightage.

Say $\gamma = 0.5$

$v_3 = 0.25 a_1 + 0.5 a_2 + a_3$

Ily $v_4 = 0.125 a_1 + 0.25 a_2 + 0.5 a_3 + a_4$

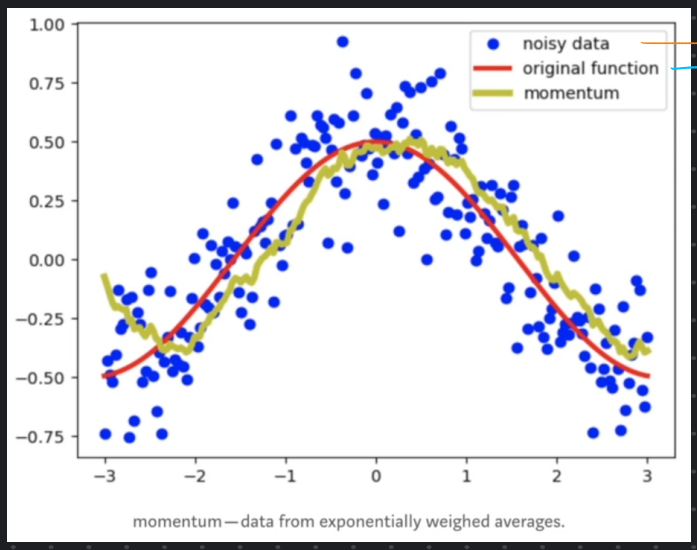
Exponentially-weighted averages:

$v_1 = 1$

$v_t = \gamma v_{t-1} + a_t$

$v_t \approx$ de-noised estimate

or $\quad v_t = a_t + \gamma v_{t-1} + \gamma^2 v_{t-2} + \gamma^3 v_{t-3} + \cdots + \gamma^{t-1} v_1$



momentum — data from exponentially weighed averages.

→ like derivatives using SGD

→ like derivatives computed using the actual data.

MB-SGD : $w_t = w_{t-1} - \eta \left(\frac{\partial L}{\partial w}\right)_{w_{t-1}}$ say $g_t = \left(\frac{\partial L}{\partial w}\right)_{w_{t-1}}$

$w_t = w_{t-1} - \eta \cdot g_t$

Exponential weighting :

$V_t = \gamma V_{t-1} + \eta g_t$ $\quad V_1 = \eta g_t$

$W_t = W_{t-1} - V_t$

case 1: $\gamma = 0$

$V_t = \eta g_t$

$w_t = w_{t-1} - \eta g_t$ } simple SGD

A good value for $\gamma$ is 0.9.

case 2: $\gamma = 0.9$

$V_t = 0.9 V_{t-1} + \eta g_t$

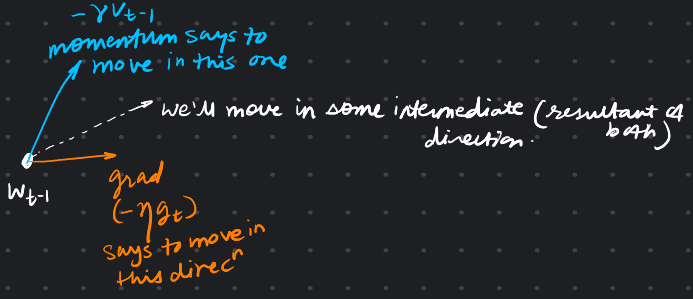
$w_t = w_{t-1} - (0.9 V_{t-1} + \eta g_t)$

this is the diff. part from the SGD update func$^n$

$w_t = w_{t-1} - [\gamma V_{t-1} + \eta g_t]$

momentum, gradient

$V_t = 1(\eta g_t) + \gamma(\eta g_{t-1}) + \gamma^2(\eta g_{t-2}) + \gamma^3(\eta g_{t-3}) + \cdots$




Momentum

SGD has trouble navigating ravines, i.e. areas where the surface curves much more steeply in one dimension than in another [1], which are common around local optima. In these scenarios, SGD oscillates across the slopes of the ravine while only making hesitant progress along the bottom towards the local optimum as in Image 2.



Image 2: SGD without momentum

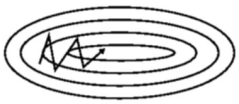
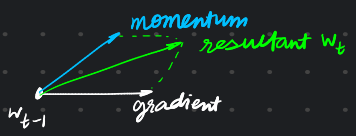


Image 3: SGD with momentum

SGD + momentum will speed up convergence.

# Nesterov accelerated gradient



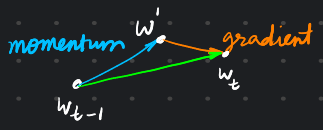

SGD + momentum

$$w_t = w_{t-1} - (\gamma v_{t-1} + \eta g_t)$$

NAG says NOT to get the resultant first. First we move towards the momentum.




now we calculate the gradient at this point and move along that direction.




$g' \neq g_t$

$$w_t = w_{t-1} - (\gamma v_{t-1} + \eta g')$$

$$g' = \left(\frac{\partial L}{\partial w}\right)_{(w_{t-1} - \gamma v_{t-1})}$$

Written as:

$$v_t = \gamma v_{t-1} + \eta g'$$

$$w_t = w_{t-1} - v_t$$

# Adagrad (adaptive gradients)

In both SGD and SGD + momentum, the learning rate (η) stays the same (something small like η=0.01). For each weight w, we use the same η.
The key idea in Adagrad is that each parameter/weight has a different η.
In the original dataset, there might be some features that are sparse (think of a BoW like representation) and some really dense (Word2Vec like).

SGD: $w_t = w_{t-1} - \eta g_t$

AdaGrad: $w_t = w_{t-1} - \eta'_t g_t$

↳ Adaptive for diff. weights and diff. iterations

$$\eta'_t = \frac{\eta}{\sqrt{\alpha_{t-1} + \epsilon}}$$

η ← constant

ε ← small value to avoid division by zero

$$\alpha_{t-1} = \sum_{i=1}^{t-1} g_i^2 \quad \text{where } g_i = \left(\frac{\partial L}{\partial w}\right)_{(w_{i-1})}$$

As $t \uparrow$, $\alpha_{t-1} \uparrow$
$\Rightarrow \eta'_t \downarrow$

learning rate decreases (for that wt) as iterations go by.

Advantages:
· No need to tune η manually for every iteration.
· If some features are sparse, AdaGrad takes care of using the appropriate learning rate as it uses the previous gradients.

Major disadvantage:
· $\alpha_{t-1}$ can get really large.

as $\alpha_{t-1} = \sum_{i=1}^{t-1} g_i^2$

so $\eta'$ will get really small.

$\Rightarrow w_t \approx w_{t-1} \Rightarrow$ slower convergence.

# Adadelta

Adagrad : $\eta'_t = \frac{\eta}{\sqrt{\alpha_{t-1} + \epsilon}}$ ; $\alpha_{t-1} = \sum_{i=1}^{t-1} g_i^2$

key idea:
Instead of adding them all up,
we'll take an exponentially decaying avg.

$w_t = w_{t-1} - \eta'_t g_t$ ;

$\eta'_t = \frac{\eta}{\sqrt{eda_t + \epsilon}}$

$eda_t = \gamma eda_{t-1} + (1-\gamma) g_t^2$ $\gamma$ → hyperparameter (something like 0.95)

For $\gamma = 0.95$

$eda_t = 0.95\, eda_{t-1} + 0.05\, g_t^2$ $eda(0) = 0$

$= 0.05\, g_t^2 + (0.95)(0.05)\, g_{t-1}^2 + (0.95)^2 eda_{t-2}$

⋮

Instead of a sum of squares, we take an exponentially decaying average of squared gradients. The denominator doesn't grow too much.

RMSProp essentially does the same thing.

# Adam: adaptive moment estimation

Adadelta → storing exponentially weighted average (eda) of gradients$^2$ → modify learning rate ($\eta_t'$)

Idea: we store eda of $g_t$ along with $g_t^2$.

---

**moments:**

mean is called the 1st order moment — so eda of $g_t$ is like mean

variance 2nd order moment — eda of $g_t^2$ is like variance.

---

$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1)\, g_t$ ——①

(mean (eda of $g_t$'s))

$\beta_1 \in [0,1]$ typically 0.9

$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$ ——②

(variance (eda of $g_t^2$'s))

$\beta_2 \in [0,1]$ typically 0.99

$\beta_1 = 0$ → Adadelta

$$\hat{m}_t = \frac{m_t}{1-(\beta_1)^t} \quad ; \quad \hat{v}_t = \frac{v_t}{1-(\beta_2)^t} \quad ; \quad w_t = w_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$\alpha$ is often 0.001

## Which algo to use when?

Among MB-SGD, momentum+SGD, NAG, Adagrad, Adadelta, RMSProp, and Adam, which to use?

- MB-SGD: use only when you have a shallow NN.
- Adagrad: esp. when sparse features
- Adam: favourite

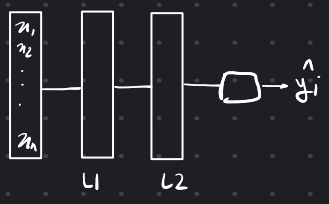
If you're stuck at a saddle point, use momentum or NAG } But they're slower.

# Gradient monitoring and clipping

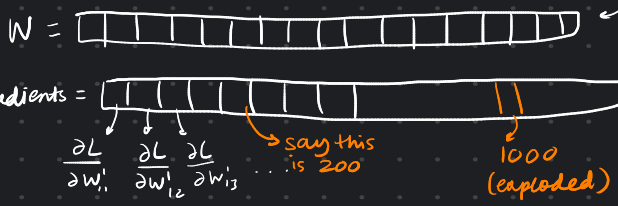
↳ we monitor gradients for each wt. and update for each epoch, wt. and layer. ← whole update term

→ To detect problems like vanishing gradient (esp. in the starting layers)

→ Exploding gradient

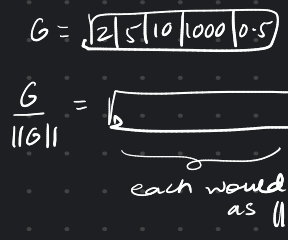



Gradient clipping : soln to exploding gradient.




L2 Norm clipping : $G_{new} = \frac{G}{\|G\|_2} * \tau$

$\tau$ : threshold.




$\|G\|_2 = \sqrt{G_1^2 + G_{12}^2 + \dots}$

new gradients $\leq \tau$

# Softmax classifier

logistic regression solves the binary classificaⁿ problem.
To use it for multiclass classificaⁿ, we use 1 vs rest. Instead we could use softmax.

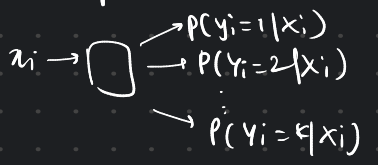
Softmax is basically logistic regression for multiclass classification.

LR:



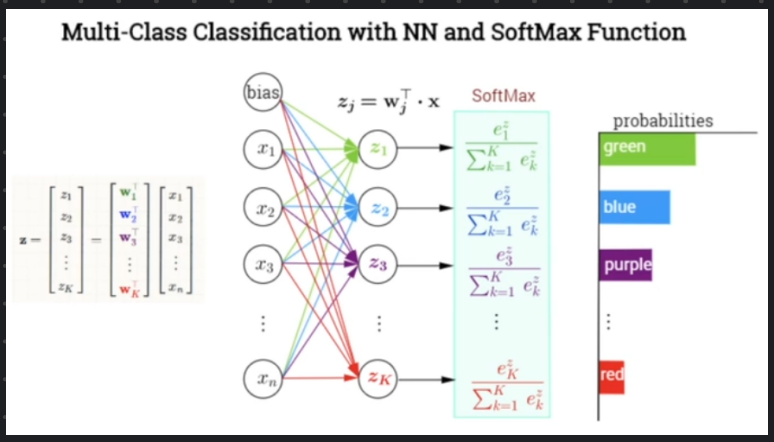
$$\hat{y}_i = P(y_i=1|x_i) = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{e^z+1}$$

Here we only need prob of 1 class, we can get the other.
In multiclass, we need all probs. or k-1 probs.




Softmax classifier (multi-class)

$D = \{x_i, y_i\}$ $y_i \in \{1, 2, 3, \cdots, k\}$




$x_{i1}$

$x_{i2}$

$x_{i3}$

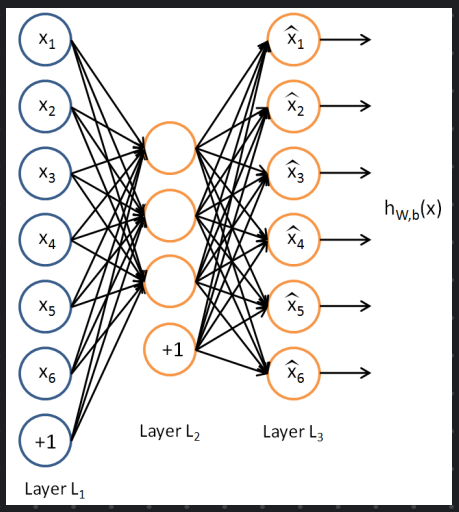
# Autoencoders

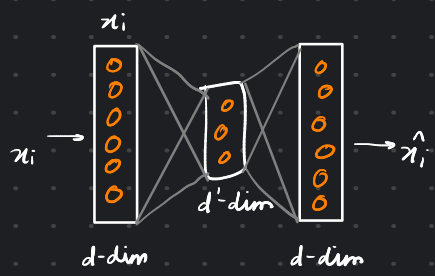
- NN which performs dimension reduction.

$D = \{x_i\}_{i=1}^n \longrightarrow D' = \{x_i'\}_{i=1}^n$

$x_i \in \mathbb{R}^d$ $\quad x_i' \in \mathbb{R}^{d'}$ s.t. $d' \ll d$

Say $x_i \in \mathbb{R}^6$, we want $x_i' \in \mathbb{R}^3$.



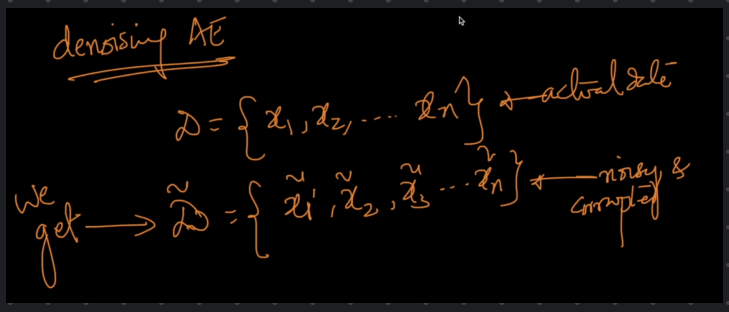




we want $\hat{x}_i \approx x_i$

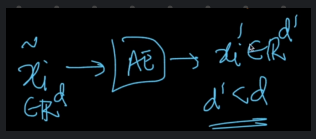
A loss function could be $L(x_i, \hat{x}_i) = \|x_i - \hat{x}_i\|^2$

Think of what it does as compression and expansion.
Output of hidden layer is the transformed data.

## Denoising AE

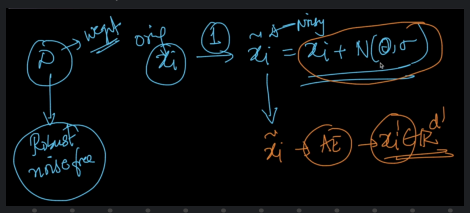



Say we get some data that's not the original data,
but data with much noise.
So we build an auto encoder on the noisy data such that




The auto encoder tries to achieve a smaller dimension of data, so whenever it finds some noise, it tries to denoise the data.
The new representation is more robust to noise.

Say we have the original data D, but we still want a more robust representation.
We'll first add some noise, maybe Gaussian noise, intentionally.
Then we encode the data using an autoencoder.

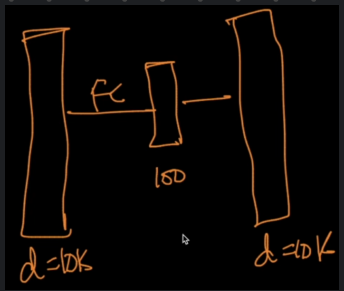



**Sparse autoencoder** [ edit ]

Autoencoders were originally invented in the 1980s; however, the initial versions were difficult to train, as the encodings have to compete to set the same small set of bits. This was solved by "sparse autoencoding". In a sparse autoencoder, there are actually more (rather than fewer) hidden units than inputs, but only a small number of the hidden units are allowed to be active at the same time.[5]

Sparsity may be achieved by additional terms in the loss function during training (by comparing the probability distribution of the hidden unit activations with some low desired value),[9] or by manually zeroing all but the few strongest hidden unit activations (referred to as a *k-sparse autoencoder*).[10]

## Sparse autoencoder




Say we have data with a huge dimension. We don't want the encoded data to be dense regardless of the original data.

We'll apply an L1 regularizer to the encoded data.
L1 regularizer creates sparsity.

### Relationship with truncated singular value decomposition (TSVD) [ edit ]

If linear activations are used, or only a single sigmoid hidden layer, then the optimal solution to an autoencoder is strongly related to principal component analysis (PCA).[12][13]

## Principal component analysis




Reconstruction of 28x28pixel images by an Autoencoder with a code size of two (two-units hidden layer) and the reconstruction from the first two Principal Components of PCA. Images come from the Fashion MNIST dataset.[28]

If linear activations are used, or only a single sigmoid hidden layer, then the optimal solution to an autoencoder is strongly related to principal component analysis (PCA).[30] [31] The weights of an autoencoder with a single hidden layer of size $p$ (where $p$ is less than the size of the input) span the same vector subspace as the one spanned by the first $p$ principal components, and the output of the autoencoder is an orthogonal projection onto this subspace. The autoencoder weights are not equal to the principal components, and are generally not orthogonal, yet the principal components may be recovered from them using the singular value decomposition.[32]

However, the potential of autoencoders resides in their non-linearity, allowing the model to learn more powerful generalizations compared to PCA, and to reconstruct the input with significantly lower information loss.[10]